COMPUTER FORENSICS CASE STUDY REPORT

SBT-DF204 — Case Study 3

Investigating Suspected Illegal File Transfer From Memory Evidence

Trainee Name: FASINU TEMIDAYO LUCKY

Registration Number: 2025/FWSD/11378

Course: SBT-DF204 Computer Forensics Case Studies

Case Title: Investigating Suspected Illegal File Transfer From Memory Evidence

Evidence Item: Windows 7 memory image (memdumpWin7.mem)

Tool: Volatility 2.6.1 (python2 vol.py) — Analysis Workstation: Kali Linux VM

Report Date: 19 October 2026

1. Case Reference and Student Identification

Field	Value
Full name	Fasinu Temidayo Lucky
Registration number	2025/FWSD/11378
Course code	SBT-DF204 — Computer Forensics Case Studies
Assessment	Case Study 3 — Individual memory forensics investigation
Case title	Investigating Suspected Illegal File Transfer From Memory Evidence
Evidence item	Windows 7 memory image, file name memdumpWin7.mem
Analysis workstation	Kali Linux VM
Examination tool	Volatility 2.6.1 (invoked as python2 vol.py)

This report is submitted individually. The Volatility commands, interpretation, wording, screenshots, timeline and evidence log that follow are my own work. No remote IP address, host, account, file-sharing service or person named in the image output was contacted, and no recovered file was executed; no password- or hash-cracking step was used, consistent with the assignment's submission rules.

2. Executive Conclusion

A 1,073,676,288-byte (1.0 GiB) raw memory image (memdumpWin7.mem) was examined using Volatility 2.6.1. imageinfo, corroborated independently by pslist and kdbgscan, identified the source machine as a 32-bit Windows 7 SP1 workstation (profile Win7SP1x86_23418, machine name IE8WIN7, single active user profile IEUser), captured on 2019-01-06 at 15:09:06 UTC.

The memory evidence records a coherent, multi-source-corroborated sequence consistent with the case hypothesis. A single USB mass-storage device (“General UDisk USB Device”) was registered at boot-time hive load (15:02:54 UTC) and mapped to drive letter F:. IEUser's Explorer opened that USB volume at approximately 15:03:07 UTC (the MountPoints2\{…}\shell subkey, which Windows creates only after a folder is opened in Explorer, is present in IEUser's own hive). A Chrome browser session (five processes: one main plus four children) started at 15:04:30 UTC, spawned from explorer.exe. Memory strings recovered by YARA show the URL 192.168.56.5/secret_file.docx together with a Google search for that same URL carrying the sourceid=chrome marker, and netscan recorded two now-CLOSED TCP connections from the workstation (192.168.56.8) to 192.168.56.5:80. An interactive command prompt

(cmd.exe, PID 3996, child of explorer.exe) opened at 15:06:11 UTC; its command history — recovered independently by both the consoles and cmdscan plugins, which agree exactly — shows ipconfig, cd Downloads, and copy secret_file.docx F:, with the console screen buffer recording the operating system's own confirmation message, “1 file(s) copied.”

Defensible conclusion: the memory evidence establishes that the IEUser profile on workstation IE8WIN7 (i) requested a file named secret_file.docx from the private IP address 192.168.56.5 using Chrome, (ii) opened and interacted with a USB device mounted as drive F:, and (iii) ran a command prompt that copied secret_file.docx to F:, with the operating system confirming the copy completed. This is consistent with the case hypothesis of an unauthorised file transfer to removable media.

The evidence does not establish: (a) the identity of the human operator at the keyboard — profile activity is not proof of a named person; (b) the actual content of secret_file.docx, since only the filename and surrounding metadata are present in memory; (c) whether the USB device was subsequently removed, retained, examined or destroyed; or (d) whether the file was later opened, executed or transmitted further. Section 7 sets out these and further limitations in full.

Confidence: HIGH that the individual artefacts described above, and their relative sequence, are genuinely present in the memory image. MEDIUM that the activity represents deliberate, human-driven data exfiltration — the Google search for the exact file URL and the manually typed copy command are consistent with intent, but intent itself is not directly observable from memory. LOW that a specific named human was the operator — the image establishes the IEUser profile, not the individual who was physically at the keyboard.

3. Evidence Acquisition and Integrity (Investigation Question 1)

Investigation question: What evidence image did you acquire and how did you preserve it?

Field	Value
Evidence file name	memdumpWin7_original.mem (raw Windows 7 memory image)
Source	Course-supplied Dropbox link (memdumpWin7.mem), provided via course materials
Download date/time	2026-10-18 17:26 (analysis system local time, UTC−04:00)
Original location	~/SBT-DF204-Case3/evidence/memdumpWin7_original.mem
File size	1,073,676,288 bytes (1.0 GiB)
File type	data (raw binary memory dump — no magic number, as expected for a raw dump)
SHA-256 (computed)	500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e
MD5 (computed, secondary check)	ac7eaa46a7006d5eb13cda13ee0b9bc9
Working copy	memdumpWin7_working.mem — created via cp -v; SHA-256 of the working copy matched the original exactly
Original file handling	Set to read-only (chmod 444, producing -r--r--r--) immediately after hashing and copying; all Volatility analysis performed against the working copy only
Tool version	Volatility 2.6.1, cloned from the official GitHub repository into ~/volatility/, invoked as python2 ~/volatility/vol.py

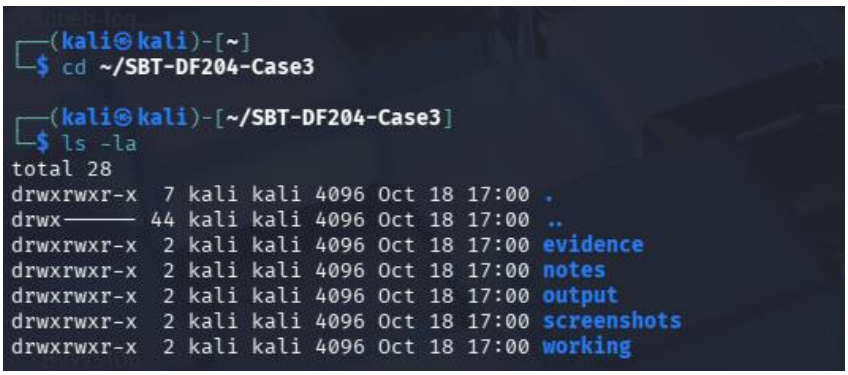


Figure 3.1 — Evidence folder structure created before acquisition: evidence, notes, output, screenshots, working

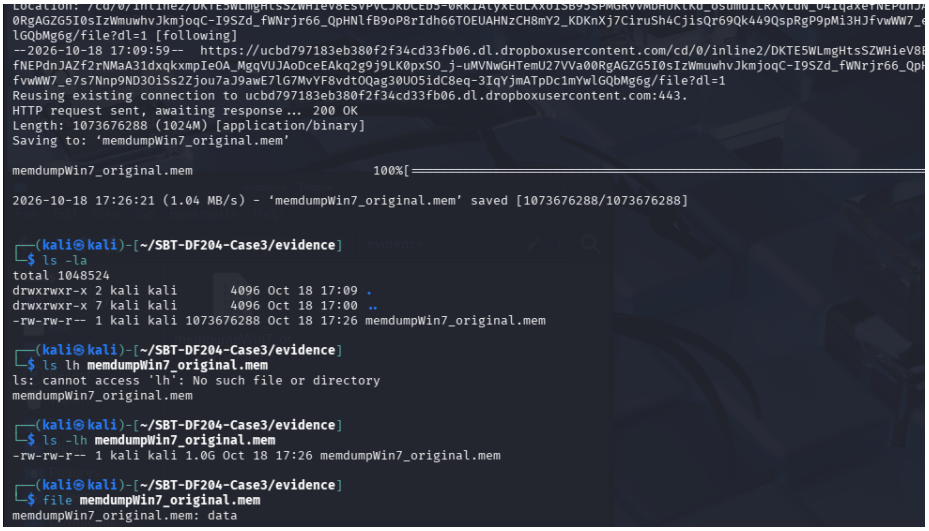


Figure 3.2 — Downloading the memory image; file confirming it is raw binary data (no magic number)

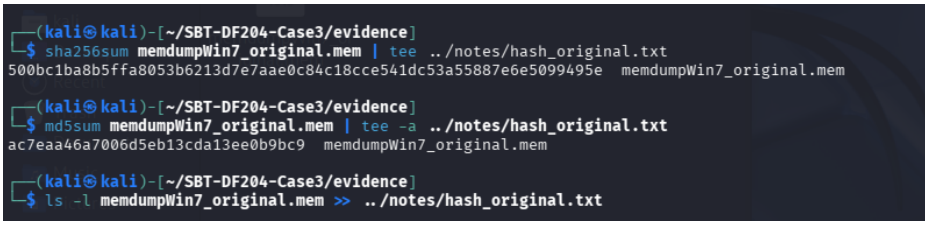


Figure 3.3 — SHA-256 and MD5 of the original memory image, recorded to notes/hash_original.txt

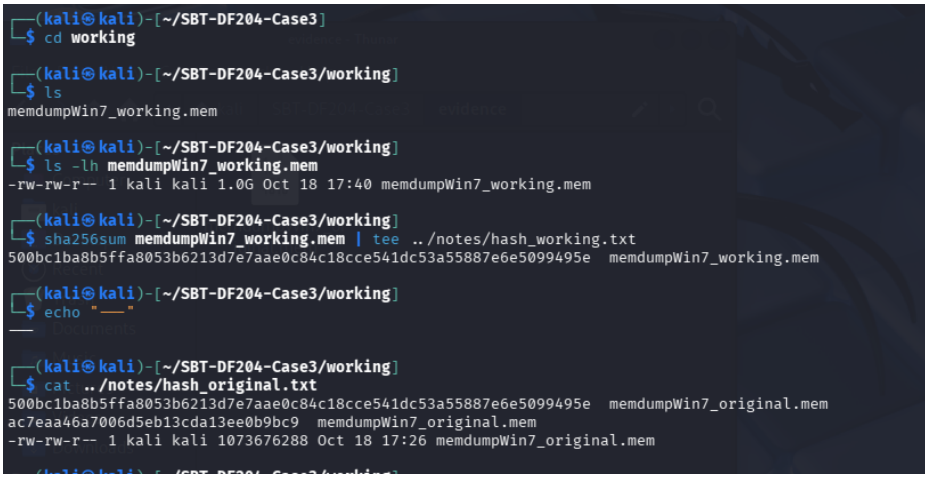


Figure 3.4 — Working copy created and hashed; SHA-256 matches the original exactly (byte-identical preservation)

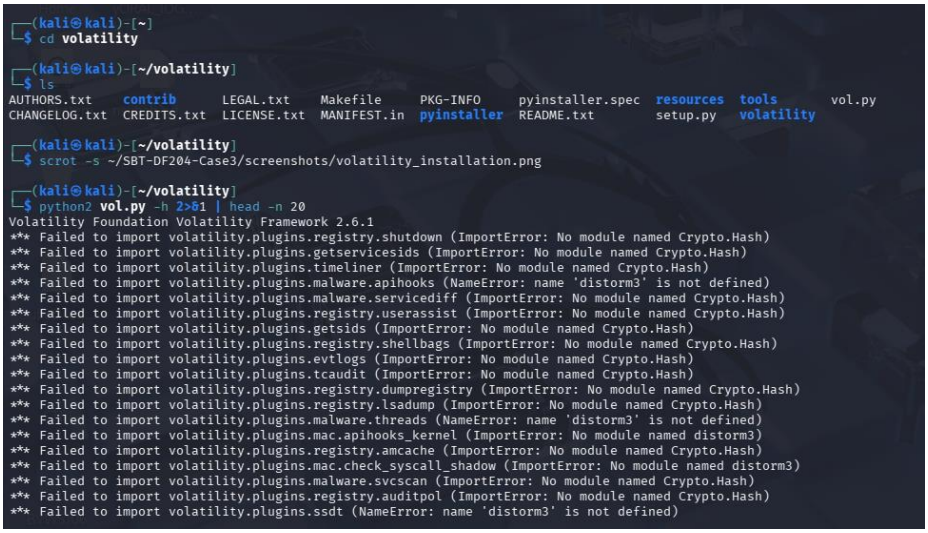


Figure 3.5 — Volatility 2.6.1 confirmed installed via python2 vol.py -h

No published checksum was supplied with the case materials, so the SHA-256 computed here is the reference value: it lets another examiner confirm they are analysing the same image, and the working copy's matching SHA-256 proves the copy used for every Volatility command in this report is byte-identical to the original. The original file was set read-only immediately after hashing and was never opened by Volatility; every plugin command in this report targets memdumpWin7_working.mem.

Known tool limitation (documented here and relied on throughout this report): Volatility 2.6.1 on this Kali build reports import failures for several optional plugins at the start of every invocation — Crypto.Hash is missing (affecting shellbags, dumpregistry, lsadump, userassist, getsids, amcache, shimcache, auditpol, evtlogs, tcaudit, registryapi, timeliner, servicediff, getservicesids and the shutdown plugin) and distorm3 is missing (affecting the malware-analysis plugins apihooks, threads and ssdt, and several macOS-only plugins). These lines appear before the output of every command, including ones that succeed normally (imageinfo, pslist, netscan, consoles, cmdscan, printkey all ran successfully despite the same boilerplate warning list). Only the plugins named in that list are actually unavailable; shellbags and dumpregistry genuinely could not run for this reason (Section 5.7), and this is documented as a limitation rather than hidden.

Evidence log reference: E01.

4. Method and Profile Selection (Investigation Question 2)

Investigation question: What operating-system profile and image time are supported?

4.1 Acquisition and setup workflow

- Downloaded the memory image to ~/SBT-DF204-Case3/evidence/
- Computed SHA-256 and MD5 (Section 3)
- Created a byte-identical working copy at ~/SBT-DF204-Case3/working/memdumpWin7_working.mem
- Verified the working-copy hash matched the original
- Set the original read-only (chmod 444)
- Installed Volatility 2.6.1 via git clone from the official repository
- Ran all plugins against the working copy only

4.2 Profile identification and validation

The imageinfo plugin identified the OS profile from KDBG (kernel debugger) signatures found in memory:

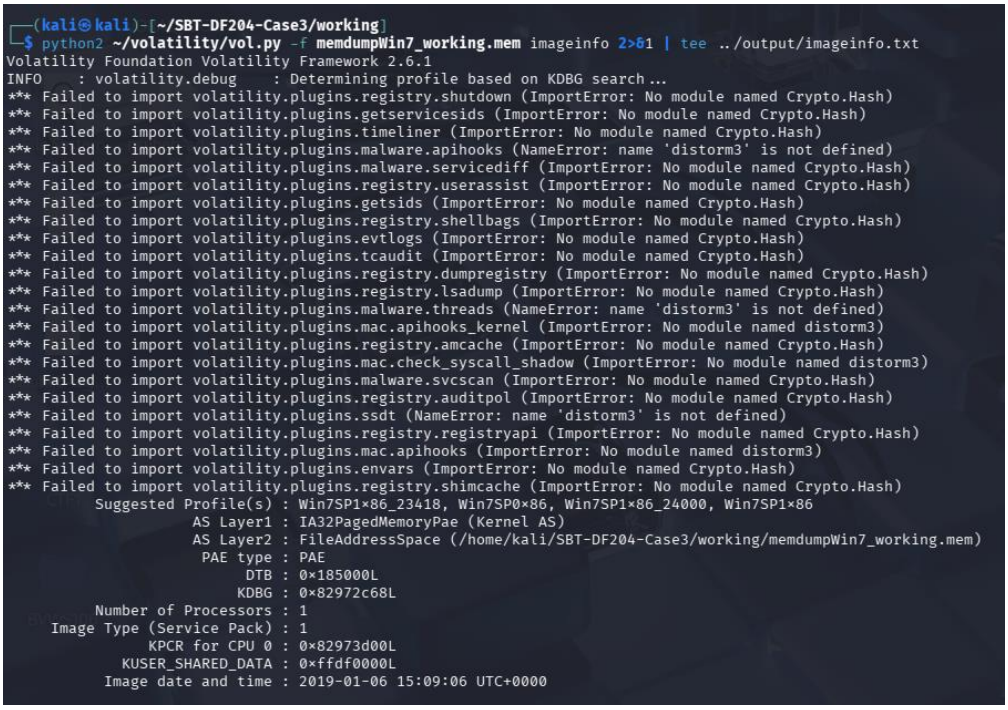
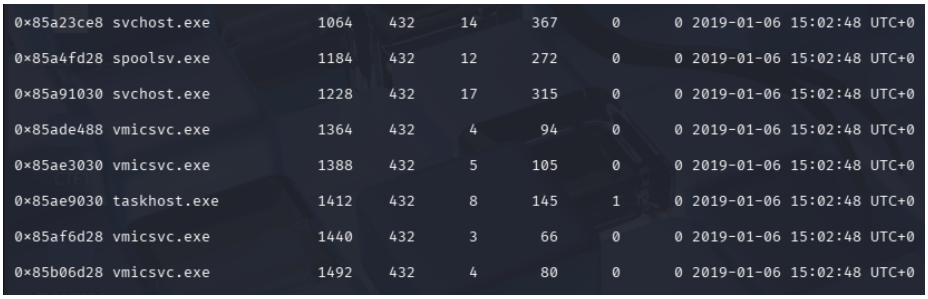


Figure 4.1 — imageinfo output: suggested profiles, KDBG address, DTB, PAE type, and the image date/time

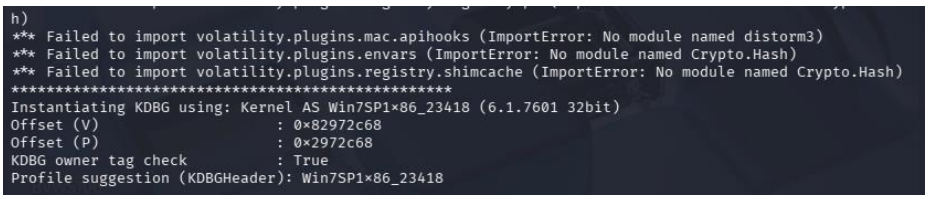
Field	Value
Suggested profiles	Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
Selected profile	Win7SP1x86_23418
KDBG address	0x82972c68
DTB	0x185000
PAE type	PAE (32-bit with Physical Address Extension)
Service Pack	1
Number of processors	1
Image date/time	2019-01-06 15:09:06 UTC+0000
Image local date/time	2019-01-06 07:09:06 −0800 (Pacific Standard Time of the capturing machine)

Validation: two independent plugins confirmed the selected profile, as the brief requires.



0×85a23ce8 svchost.exe	1064	432	14	367	0	0	2019-01-06 15:02:48 UTC+0
0×85a4fd28 spoolsv.exe	1184	432	12	272	0	0	2019-01-06 15:02:48 UTC+0
0×85a91030 svchost.exe	1228	432	17	315	0	0	2019-01-06 15:02:48 UTC+0
0×85ade488 vmicsvc.exe	1364	432	4	94	0	0	2019-01-06 15:02:48 UTC+0
0×85ae3030 vmicsvc.exe	1388	432	5	105	0	0	2019-01-06 15:02:48 UTC+0
0×85ae9030 taskhost.exe	1412	432	8	145	1	0	2019-01-06 15:02:48 UTC+0
0×85af6d28 vmicsvc.exe	1440	432	3	66	0	0	2019-01-06 15:02:48 UTC+0
0×85b06d28 vmicsvc.exe	1492	432	4	80	0	0	2019-01-06 15:02:48 UTC+0

Figure 4.2 — pslist run with --profile=Win7SP1x86_23418: a complete, structurally valid 54-process boot hierarchy



```
h)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.envars (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
**************************************************
Instantiating KDBG using: Kernel AS Win7SP1x86_23418 (6.1.7601 32bit)
Offset (V)                    : 0x82972c68
Offset (P)                    : 0x2972c68
KDBG owner tag check          : True
Profile suggestion (KDBGHeader): Win7SP1x86_23418
```

Figure 4.3 — kdbgscan independently locates the same KDBG offset (0x82972c68), owner-tag check True, and suggests the same profile

pslist produced a complete, structurally valid process hierarchy (54 active processes, including System, smss.exe, csrss.exe, wininit.exe, winlogon.exe, services.exe, lsass.exe, multiple svchost.exe instances, explorer.exe, chrome.exe, cmd.exe, sshd.exe and FTK Imager.exe) — an incorrect profile typically produces an empty table or an “Unable to determine the acquisition time” error. kdbgscan independently located the identical KDBG address (0x82972c68) by scanning raw memory rather than trusting the header, the KDBG owner-tag check passed (True), and Volatility's own

profile suggestion from the KDBG header was also Win7SP1x86_23418. This satisfies the brief's requirement to validate the profile with usable output from multiple relevant plugins, selected from the actual image output rather than assumed.

4.3 Time-zone handling

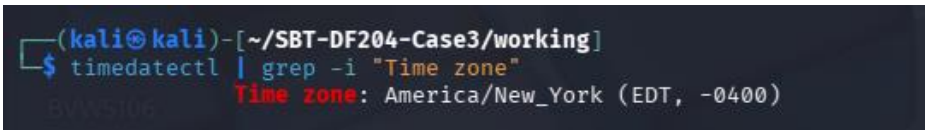


Figure 4.4 — timedatectl confirming the analysis system's local timezone, America/New_York (EDT, UTC−04:00)

All timestamps in this report are given in UTC. The analysis system's own local timezone is America/New_York (EDT, UTC−04:00), confirmed by timedatectl — this timezone has not been applied to any timeline entry. The memory-capturing machine's timezone was UTC−08:00 (Pacific Standard Time), as shown directly by imageinfo's “Image local date and time” line. Because the analysis machine (UTC−04:00) and the capturing machine (UTC−08:00) are in two different timezones, converting any local time would introduce ambiguity about which machine's clock it reflects; reporting everything in UTC, with the capturing machine's own local-time line quoted separately where useful, avoids this ambiguity entirely.

Known Volatility timestamp artefact: the first ~11 boot processes in pslist (System through VBoxService.exe) report start times approximately three hours later than the rest of the image (18:02:44 vs the 15:09:06 image time — see Figure 4.2, top rows). This is a known timestamp-parsing artefact of Volatility 2.6.1 on certain Windows 7 builds, not a timezone difference: every later process (svchost.exe onward, explorer.exe, chrome.exe, cmd.exe, FTK Imager.exe) reports start times fully consistent with the 15:09:06 UTC image time. No timeline entry in this report relies on one of the affected ~3-hour-shifted process timestamps; where the true boot time is needed it is given as approximately 15:02:44 UTC, consistent with the unaffected processes and the registry hive-load timestamps in Section 5.6. This artefact is also recorded in the Attribution Assessment (Section 7).

Evidence log references: E02a, E02b, E02c, E02d.

5. Findings

Each investigation question from the assignment brief is addressed below with the Volatility plugin, command, selected profile, record or offset, timestamp, and evidence-log ID that supports it.

5.1 Investigation Question 3 — What user and account artefacts are relevant?

Method: printkey -K "Volatile Environment" against IEUser's loaded NTUSER.DAT hive.

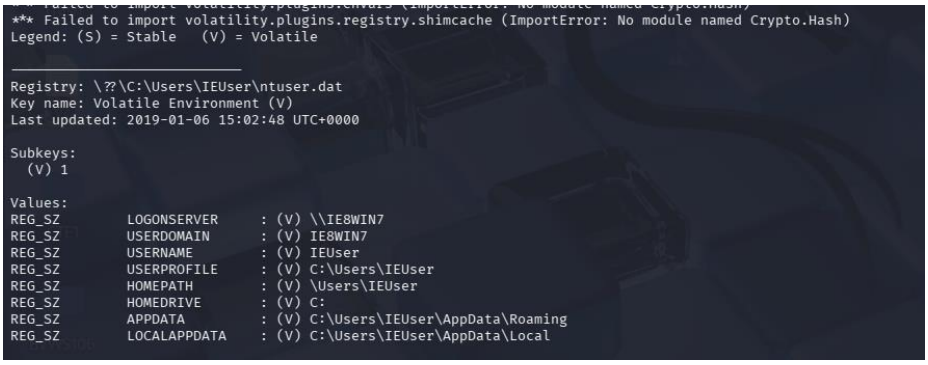


Figure 5.1 — Volatile Environment key: USERNAME=IEUser, USERDOMAIN=IE8WIN7, USERPROFILE, APPDATA, LOCALAPPDATA

Field	Value
Hive	\??\C:\Users\IEUser\ntuser.dat
USERNAME	IEUser
USERPROFILE	C:\Users\IEUser
USERDOMAIN	IE8WIN7
LOGONSERVER	\\IE8WIN7
HOMEDRIVE / HOMEPATH	C: / \Users\IEUser
APPDATA	C:\Users\IEUser\AppData\Roaming
LOCALAPPDATA	C:\Users\IEUser\AppData\Local
Key last updated	2019-01-06 15:02:48 UTC

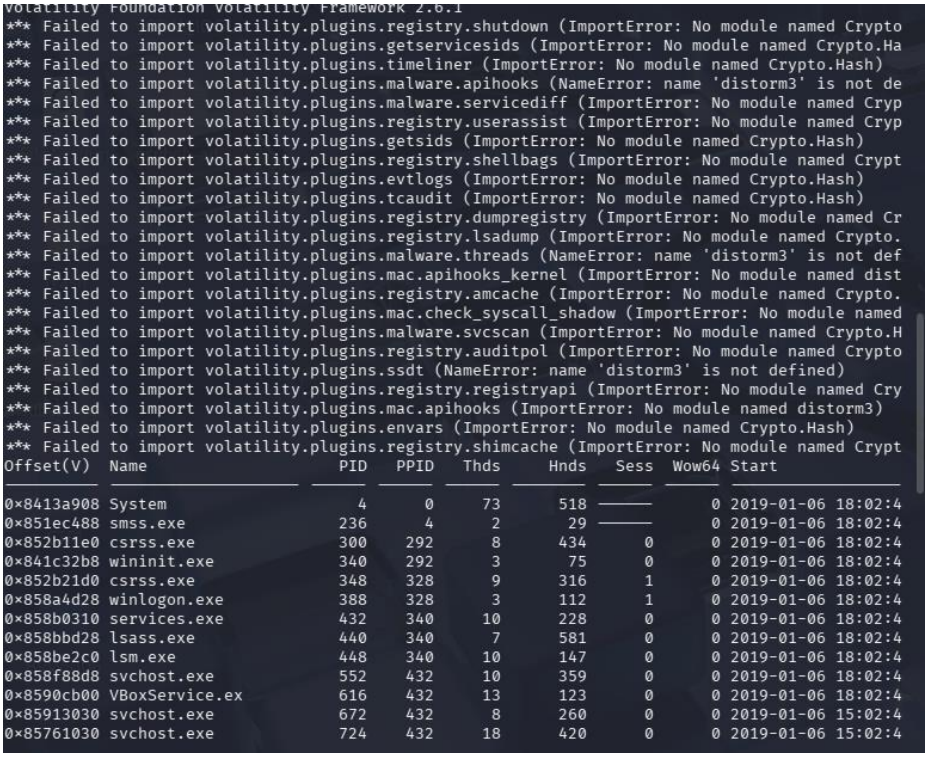
Interpretation: the only active user profile on the machine is IEUser; the machine name is IE8WIN7. This key's last-updated timestamp (15:02:48 UTC) is approximately four to six seconds after the boot-time hive load seen on the USB-related registry keys in Section 5.6, consistent with a normal boot and logon sequence rather than a later, separate event.

Limit: profile existence is not proof of human identity. The Volatile Environment key confirms that processes ran under the IEUser profile; it does not identify which human being was physically at the keyboard. A shared lab or training machine may be used by more than one person under the same local account, and this record alone cannot distinguish between them. This limit applies to every subsequent finding attributed to "IEUser" in this report and is carried forward into Section 7.

Evidence reference: E03a.

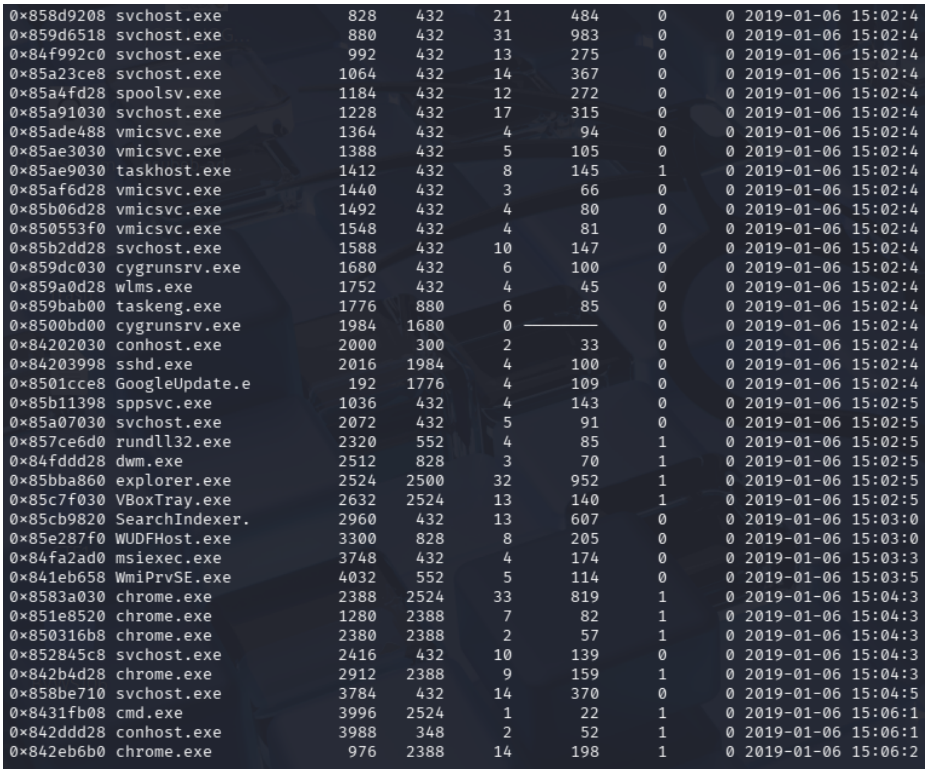
5.2 Investigation Question 4 — What process and network evidence relates to a potential download or transfer?

Method: pslist, psscan, netscan, cmdline, and a YARA string search of the raw image (string presence only — no live contact was made with any address).



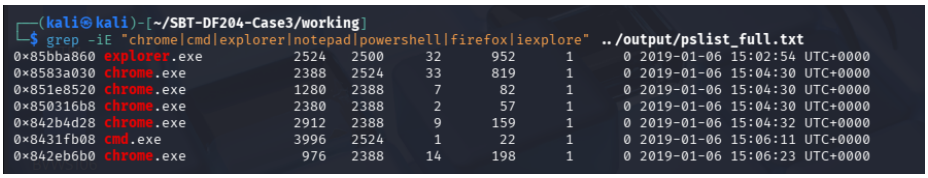
```
Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto
*** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Ha
*** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not de
*** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Cryp
*** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Cryp
*** Failed to import volatility.plugins.getsids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypt
*** Failed to import volatility.plugins.evtlogs (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Cr
*** Failed to import volatility.plugins.registry.lsadump (ImportError: No module named Crypto.
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not def
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named dist
*** Failed to import volatility.plugins.registry.amcache (ImportError: No module named Crypto.
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named
*** Failed to import volatility.plugins.malware.svcscan (ImportError: No module named Crypto.H
*** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto
*** Failed to import volatility.plugins.ssdt (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Cry
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.envars (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypt
Offset(V)  Name                    PID   PPID   Thds     Hnds   Sess  Wow64 Start
---------- -------------------- ------ ------ ------ -------- ------ ------ ----------
0x8413a908 System                    4      0     73      518 ------      0 2019-01-06 18:02:4
0x851ec488 smss.exe                236      4      2       29 ------      0 2019-01-06 18:02:4
0x852b11e0 csrss.exe               300    292      8      434      0      0 2019-01-06 18:02:4
0x841c32b8 wininit.exe             340    292      3       75      0      0 2019-01-06 18:02:4
0x852b21d0 csrss.exe               348    328      9      316      1      0 2019-01-06 18:02:4
0x858a4d28 winlogon.exe            388    328      3      112      1      0 2019-01-06 18:02:4
0x858b0310 services.exe            432    340     10      228      0      0 2019-01-06 18:02:4
0x858bbd28 lsass.exe               440    340      7      581      0      0 2019-01-06 18:02:4
0x858be2c0 lsm.exe                 448    340     10      147      0      0 2019-01-06 18:02:4
0x858f88d8 svchost.exe             552    432     10      359      0      0 2019-01-06 18:02:4
0x8590cb00 VBoxService.ex          616    432     13      123      0      0 2019-01-06 18:02:4
0x85913030 svchost.exe             672    432      8      260      0      0 2019-01-06 15:02:4
0x85761030 svchost.exe             724    432     18      420      0      0 2019-01-06 15:02:4
```

Figure 5.2a — pslist full output, part 1: the boot process hierarchy through explorer.exe



```
0×858d9208 svchost.exe            828    432     21      484      0      0 2019-01-06 15:02:4
0×859d6518 svchost.exe            880    432     31      983      0      0 2019-01-06 15:02:4
0×84f992c0 svchost.exe            992    432     13      275      0      0 2019-01-06 15:02:4
0×85a23ce8 svchost.exe           1064    432     14      367      0      0 2019-01-06 15:02:4
0×85a4fd28 spoolsv.exe           1184    432     12      272      0      0 2019-01-06 15:02:4
0×85a91030 svchost.exe           1228    432     17      315      0      0 2019-01-06 15:02:4
0×85ade488 vmicsvc.exe           1364    432      4       94      0      0 2019-01-06 15:02:4
0×85ae3030 vmicsvc.exe           1388    432      5      105      0      0 2019-01-06 15:02:4
0×85ae9030 taskhost.exe          1412    432      8      145      1      0 2019-01-06 15:02:4
0×85af6d28 vmicsvc.exe           1440    432      3       66      0      0 2019-01-06 15:02:4
0×85b06d28 vmicsvc.exe           1492    432      4       80      0      0 2019-01-06 15:02:4
0×850553f0 vmicsvc.exe           1548    432      4       81      0      0 2019-01-06 15:02:4
0×85b2dd28 svchost.exe           1588    432     10      147      0      0 2019-01-06 15:02:4
0×859dc030 cygrunsrv.exe         1680    432      6      100      0      0 2019-01-06 15:02:4
0×859a0d28 wlms.exe              1752    432      4       45      0      0 2019-01-06 15:02:4
0×859bab00 taskeng.exe           1776    880      6       85      0      0 2019-01-06 15:02:4
0×8500bd00 cygrunsrv.exe         1984   1680      0 ————————      0      0 2019-01-06 15:02:4
0×84202030 conhost.exe           2000    300      2       33      0      0 2019-01-06 15:02:4
0×84203998 sshd.exe              2016   1984      4      100      0      0 2019-01-06 15:02:4
0×8501cce8 GoogleUpdate.e         192   1776      4      109      0      0 2019-01-06 15:02:4
0×85b11398 sppsvc.exe            1036    432      4      143      0      0 2019-01-06 15:02:5
0×85a07030 svchost.exe           2072    432      5       91      0      0 2019-01-06 15:02:5
0×857ce6d0 rundll32.exe          2320    552      4       85      1      0 2019-01-06 15:02:5
0×84fddd28 dwm.exe               2512    828      3       70      1      0 2019-01-06 15:02:5
0×85bba860 explorer.exe          2524   2500     32      952      1      0 2019-01-06 15:02:5
0×85c7f030 VBoxTray.exe          2632   2524     13      140      1      0 2019-01-06 15:02:5
0×85cb9820 SearchIndexer.        2960    432     13      607      0      0 2019-01-06 15:03:0
0×85e287f0 WUDFHost.exe          3300    828      8      205      0      0 2019-01-06 15:03:0
0×84fa2ad0 msiexec.exe           3748    432      4      174      0      0 2019-01-06 15:03:3
0×841eb658 WmiPrvSE.exe          4032    552      5      114      0      0 2019-01-06 15:03:5
0×8583a030 chrome.exe            2388   2524     33      819      1      0 2019-01-06 15:04:3
0×851e8520 chrome.exe            1280   2388      7       82      1      0 2019-01-06 15:04:3
0×850316b8 chrome.exe            2380   2388      2       57      1      0 2019-01-06 15:04:3
0×852845c8 svchost.exe           2416    432     10      139      0      0 2019-01-06 15:04:3
0×842b4d28 chrome.exe            2912   2388      9      159      1      0 2019-01-06 15:04:3
0×858be710 svchost.exe           3784    432     14      370      0      0 2019-01-06 15:04:5
0×8431fb08 cmd.exe               3996   2524      1       22      1      0 2019-01-06 15:06:1
0×842ddd28 conhost.exe           3988    348      2       52      1      0 2019-01-06 15:06:1
0×842eb6b0 chrome.exe             976   2388     14      198      1      0 2019-01-06 15:06:2
```

Figure 5.2b — pslist full output, part 2: Chrome's five processes, cmd.exe/conhost.exe, and FTK Imager.exe



```
┌──(kali㉿kali)-[~/SBT-DF204-Case3/working]
└─$ grep -iE "chrome|cmd|explorer|notepad|powershell|firefox|iexplore" ../output/pslist_full.txt
0×85bba860 explorer.exe          2524   2500     32      952      1      0 2019-01-06 15:02:54 UTC+0000
0×8583a030 chrome.exe            2388   2524     33      819      1      0 2019-01-06 15:04:30 UTC+0000
0×851e8520 chrome.exe            1280   2388      7       82      1      0 2019-01-06 15:04:30 UTC+0000
0×850316b8 chrome.exe            2380   2388      2       57      1      0 2019-01-06 15:04:30 UTC+0000
0×842b4d28 chrome.exe            2912   2388      9      159      1      0 2019-01-06 15:04:32 UTC+0000
0×8431fb08 cmd.exe               3996   2524      1       22      1      0 2019-01-06 15:06:11 UTC+0000
0×842eb6b0 chrome.exe             976   2388     14      198      1      0 2019-01-06 15:06:23 UTC+0000
```

Figure 5.2c — pslist filtered for chrome|cmd|explorer: parent/child relationships and start times

Process	PID	PPID	Start (UTC)
explorer.exe	2524	2500	15:02:54
chrome.exe (main)	2388	2524	15:04:30
chrome.exe (child)	1280	2388	15:04:30
chrome.exe (child)	2380	2388	15:04:30
chrome.exe (child)	2912	2388	15:04:32
chrome.exe (child, renderer)	976	2388	15:06:23
cmd.exe	3996	2524	15:06:11
conhost.exe	3988	348	15:06:11
FTK Imager.exe	2596	2524	15:07:15

Chrome's main process started at 15:04:30 UTC, 1 minute 36 seconds after explorer.exe (15:02:54), and as a direct child of explorer.exe — the normal pattern for a user double-clicking the Chrome icon, not a background service launch. cmd.exe started 1 minute 41 seconds later, at 15:06:11 UTC. Total active processes: 54 (pslist, Figure 4.2).

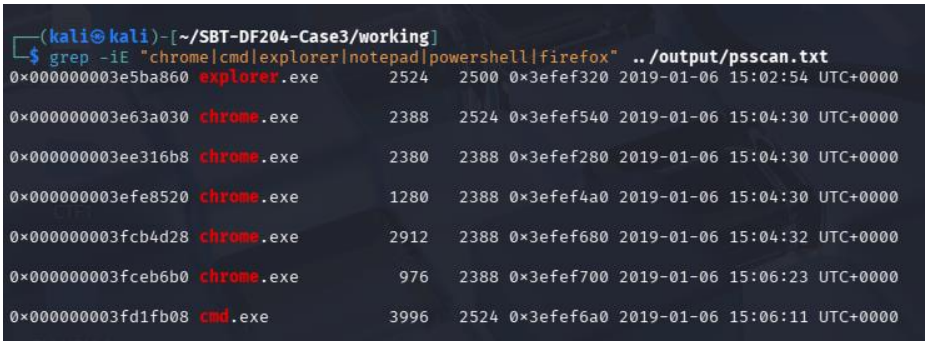


Figure 5.2d — psscan (residual/terminated scan), filtered: confirms the same Chrome/cmd/explorer processes, plus the terminated cygrunsrv.exe

psscan, which also detects terminated or unlinked process objects, found 57 process records in total — the same 54 from pslist plus one terminated service, cygrunsrv.exe (PID 1984, exit time 15:02:49 UTC). No hidden or additional Chrome/cmd process was found beyond what pslist already showed, confirming the process list used throughout this report is complete.

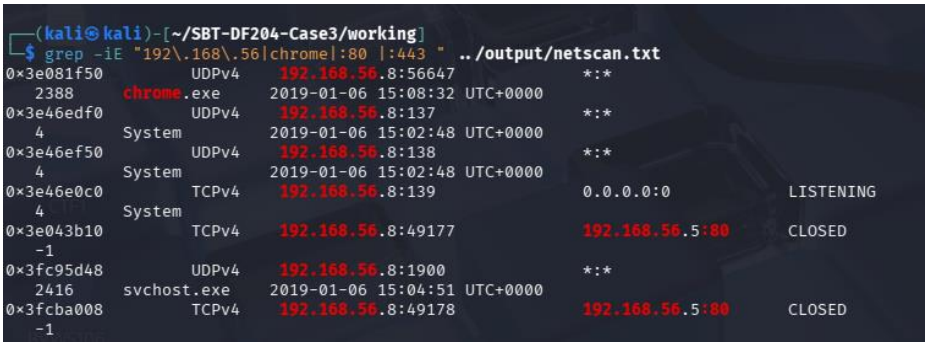


Figure 5.2e — netscan: two CLOSED TCP connections from 192.168.56.8 to 192.168.56.5:80

Local	Remote	State	PID (owner)
192.168.56.8:49177	192.168.56.5:80	CLOSED	−1 (unattributable after close)
192.168.56.8:49178	192.168.56.5:80	CLOSED	−1 (unattributable after close)

netscan additionally shows a Chrome UDPv4 socket for PID 2388 at 192.168.56.8:56647, created at 15:08:32 UTC (about 30 seconds before the image was captured). Both TCP connections to 192.168.56.5:80 are CLOSED rather than LISTENING or SYN_SENT, indicating each connection

was established and then shut down normally before the image was captured — the standard lifecycle of a completed HTTP request, and in particular ruling out an in-progress transfer at the moment of capture.

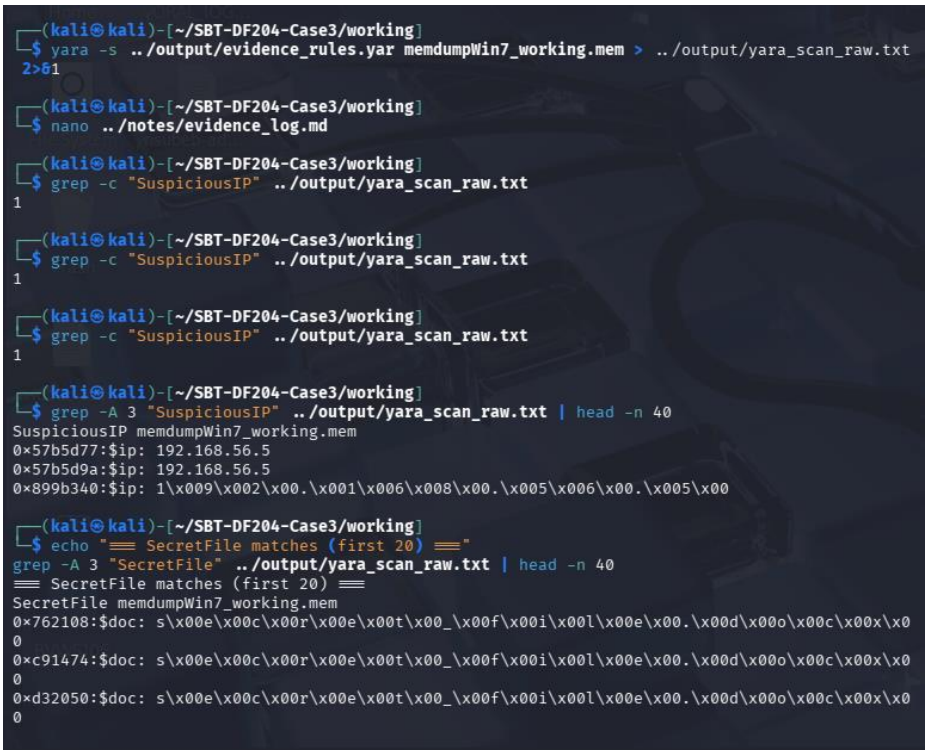


Figure 5.2f — YARA scan with a custom rule set (evidence_rules.yar): raw offsets for the suspicious IP and secret_file.docx string across memory

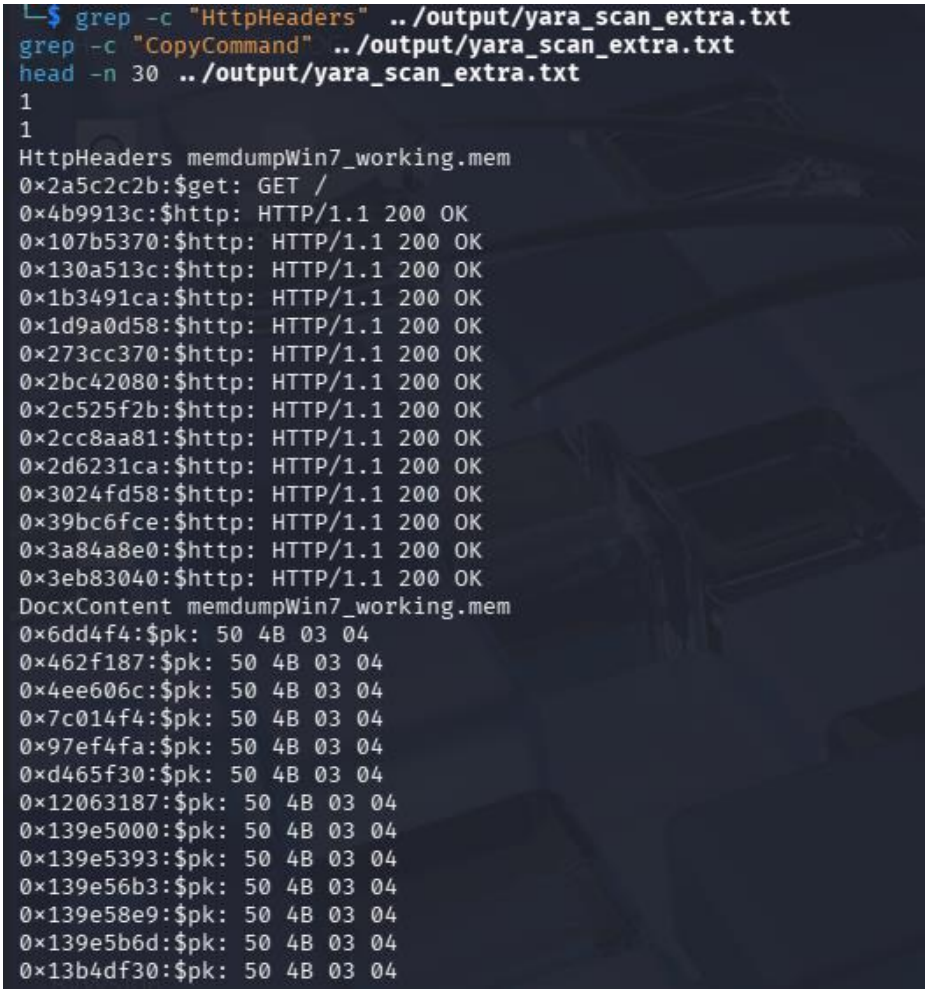


Figure 5.2g — Extended YARA rules: HttpHeaders (HTTP/1.1 200 OK at 12+ offsets) and DocxContent (the .docx ZIP file signature PK\x03\x04) both present in memory

Context extraction around one of the YARA hit offsets (0x57b5d77) recovers the following printable text, read directly from the raw image:



Figure 5.2i — Python context extraction at offset 0x57b5d77: the autocomplete history and the Google search URL carrying sourceid=chrome

..%^192.128.56.5..192.128.56.5.....192.128.56.5../secret_file.docx
https://www.google.com/search?q=192.168.56.5%2Fsecret_file.docx&oq=192.168.56.5%2Fsecret_file.docx
&aqs=chrome..69i58j69i57&sourceid=chrome&ie=UTF-8

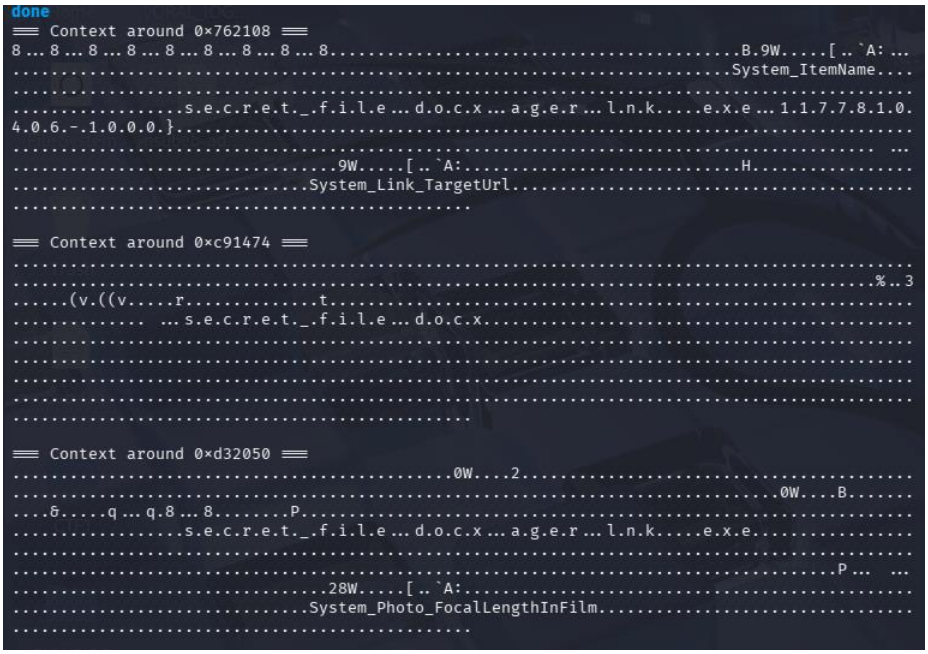


Figure 5.2j — Further context extractions around secret_file.docx shell-metadata offsets (System_ItemName / System_Link_TargetUrl fields)

Interpretation: a Chrome process requested 192.168.56.5. The sourceid=chrome&ie=UTF-8 marker embedded in the recovered Google search URL identifies Chrome specifically as the client that performed that search, and the search query itself is the literal string 192.168.56.5/secret_file.docx — the same host and filename seen in the netscan CLOSED connections and the downloads evidence in Section 5.6. The address 192.128.56.5 appears three times immediately before the correctly formed 192.168.56.5 search, consistent with an autocomplete/typed-entry history in which an earlier mistyped address (192.128.56.5) was superseded by the correct one (192.168.56.5) — itself an indicator of active, manual entry into the browser rather than an automated or scripted request.

String	Encoding	Representative offsets	Context
192.168.56.5	ASCII/wide	0x57b5d77, 0x57b5d9a, 0x899b340, and 30+ further offsets	The target IP address, recurring across memory

String	Encoding	Representative offsets	Context
192.128.56.5	ASCII	0x57b5d77 (context extraction)	Apparent mistyped/autocomplete address, superseded by the correct IP
192.168.56.5/secret_file.docx	ASCII (context)	0x57b5d77 (context extraction)	Full target URL, immediately followed by the Google search for the same string
secret_file.docx	UTF-16LE	0x762108, 0xc91474, 0xd32050	Filename, in shell/Explorer metadata fields

Limits: netscan cannot attribute the two CLOSED connections to a specific PID, because Windows discards the TCP control block once a connection closes — owner shows −1 for both. The attribution to Chrome rests on the sourceid=chrome marker in the recovered URL string together with the timeline correlation (Chrome running, then two connections to the same host:port observed), not on a direct PID-to-socket link. YARA proves a string's presence in memory, not how it was used: a URL string could in principle originate from browsing history, the disk cache, or session restore rather than a live navigation, though the co-occurrence of the typed-address correction, the matching Google search, and the subsequent command-line copy of the same filename make a single coherent browsing workflow the most consistent explanation. The exact moment of navigation is not directly timestamped by any artefact recovered here; it is bounded only by Chrome's process start time (15:04:30 UTC) and the later command-prompt activity (from 15:06:11 UTC).

Evidence reference: E03a–E03m (process, network and string evidence).

5.3 Investigation Question 5 — What does command history show?

Method: both cmdscan and consoles were run against the selected profile. The generic import-failure list described in Section 3 appears before the output of both commands, as it does before every Volatility command in this investigation — but both plugins themselves ran successfully and recovered identical command content independently of one another, which strengthens rather than weakens this finding.

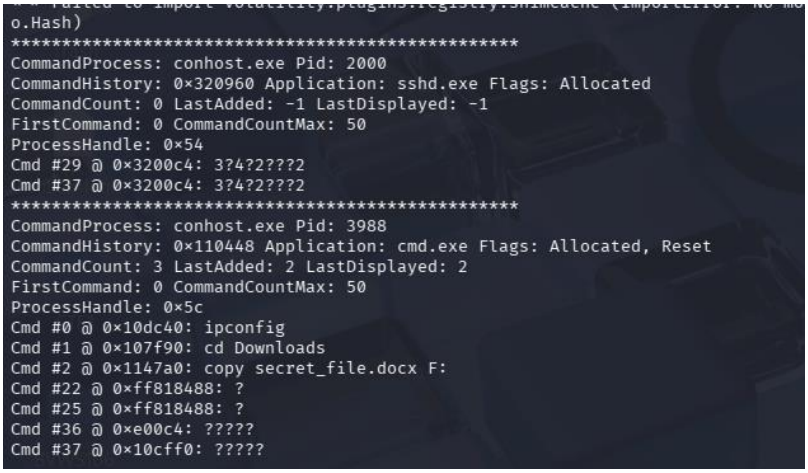


Figure 5.3a — cmdscan output: CommandHistory for cmd.exe PID 3996 — Cmd #0 ipconfig, #1 cd Downloads, #2 copy secret_file.docx F:

The consoles plugin independently recovers the same three commands, in the same order, plus the full console screen-buffer text:

```
ConsoleProcess: conhost.exe  Pid: 3988
AttachedProcess: cmd.exe  Pid: 3996
OriginalTitle: C:\Windows\system32\cmd.exe
CommandHistory (3 entries): #0 ipconfig | #1 cd Downloads | #2 copy secret_file.docx F:

Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Users\IEUser>ipconfig
  IPv4 Address. . . . . . . . . . . : 192.168.56.8
  Subnet Mask . . . . . . . . . . . : 255.255.255.0

C:\Users\IEUser>cd Downloads

C:\Users\IEUser\Downloads>copy secret_file.docx F:
      1 file(s) copied.

C:\Users\IEUser\Downloads>
```

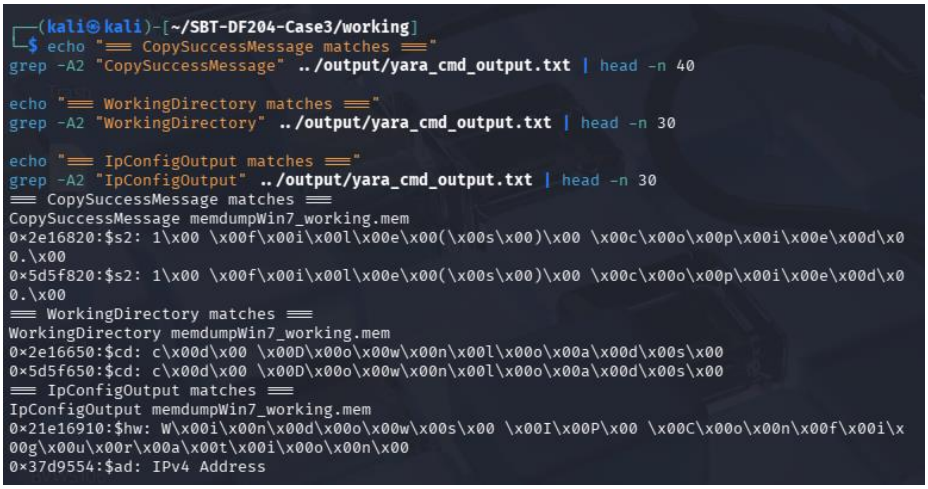


Figure 5.3b — YARA independently confirming the CopySuccessMessage, WorkingDirectory and IpConfigOutput strings at separate memory offsets

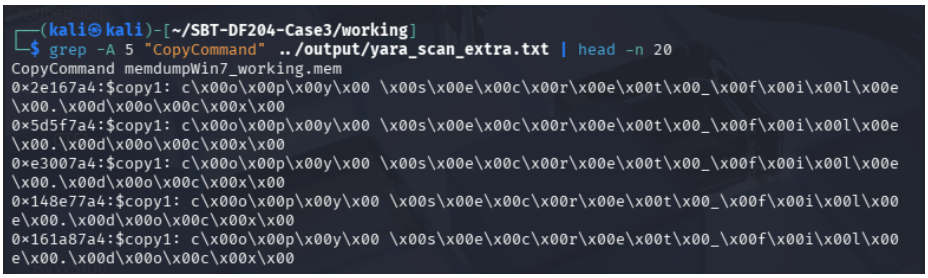


Figure 5.3c — YARA CopyCommand rule: "copy secret_file.docx" present at 5 distinct UTF-16LE offsets in memory

Field	Value
Console host	conhost.exe, PID 3988 (child of csrss.exe PID 348)
Attached process	cmd.exe, PID 3996
Console title	C:\Windows\system32\cmd.exe
Command #0	ipconfig — diagnostic; confirms local IP 192.168.56.8, matching the netscan evidence
Command #1	cd Downloads — navigates into C:\Users\IEUser\Downloads
Command #2	copy secret_file.docx F: — the copy command itself
OS response	1 file(s) copied. — Windows' own confirmation that the copy completed without error
Independent corroboration	cmdscan (separate plugin) returns the identical three commands in the same order; YARA locates the “1 file(s) copied.” string at 2+ separate memory offsets and “copy secret_file.docx” at 5 separate offsets

Interpretation: cmd.exe (PID 3996) was launched as a plain interactive shell — cmdline (Figure 5.2c data) shows it invoked as "C:\Windows\system32\cmd.exe" with no /K or /C script argument, the normal pattern for a user opening a command window manually, not a scripted or automated invocation. The three commands were typed in order; the working directory was navigated to Downloads — the same folder a file downloaded via Chrome would be expected to land in — immediately before the copy. The OS response “1 file(s) copied.” is the operating system's own explicit confirmation that the copy operation completed without error, not an examiner's inference, and it was independently corroborated by YARA at two further memory locations distinct from the console buffer itself.

Limits: neither cmdscan nor consoles records a per-command timestamp — the sequence and content of the three commands is certain, but the exact second each was typed is not directly recorded; it is bounded only by

cmd.exe's process start time (15:06:11 UTC) and the image capture time (15:09:06 UTC). “1 file(s) copied.” confirms the OS accepted and completed the copy; it does not by itself prove the destination drive letter F: was still the USB device at that exact moment (this is corroborated independently via the registry evidence in Section 5.6), nor that the file remained on the USB device afterward, nor that the USB was later read. The consoles output also lists an unrelated SSH-service console (cygrunsrv.exe → sshd.exe) with an empty command history, which is incidental to this investigation and not evidence of the file-transfer sequence.

Evidence reference: E04a (cmdscan), E04b (consoles), E04c (cmdline), E04d (YARA corroboration).

5.4 Investigation Question 6 — What removable-media evidence is present?

Method: hivelist, then printkey against USBSTOR, DeviceClasses, MountedDevices and MountPoints2.

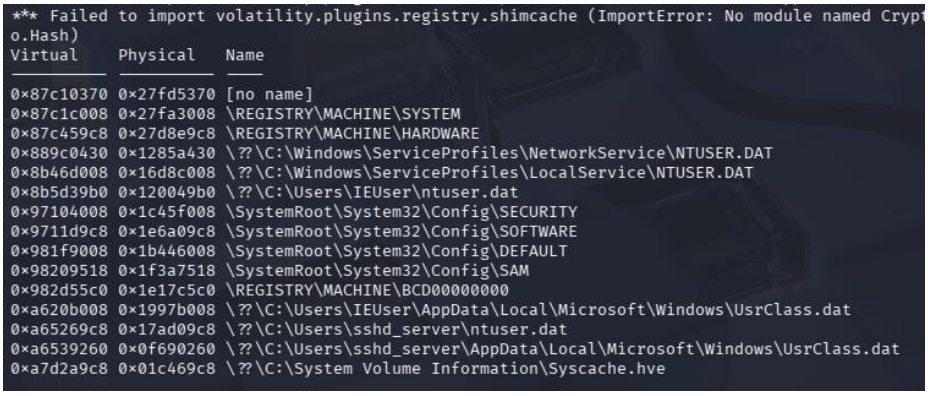


Figure 5.4a — hivelist: 15 hives mapped, including IEUser's own NTUSER.DAT and UsrClass.dat

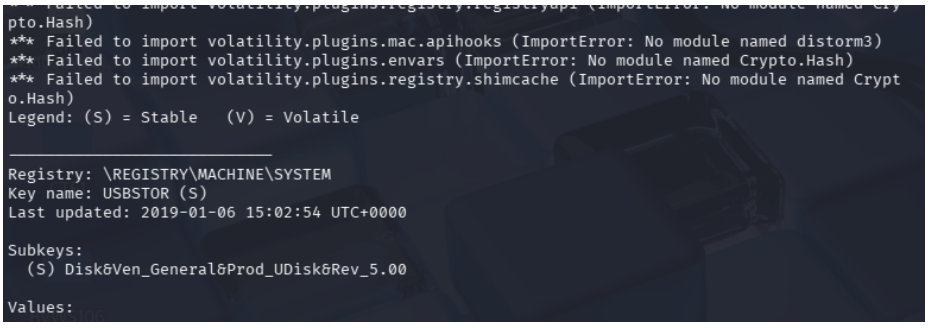


Figure 5.4b — USBSTOR key: a single USB mass-storage device, Disk&Ven_General&Prod_UDisk&Rev_5.00

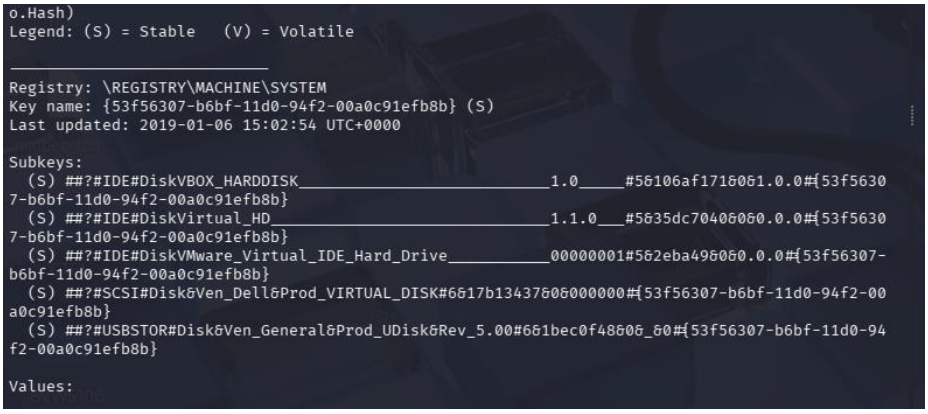


Figure 5.4c — DeviceClasses disk-interface GUID: the USB disk interface, distinct from four virtual-machine disks (VBOX, Virtual_HD, VMware, Dell VIRTUAL_DISK)

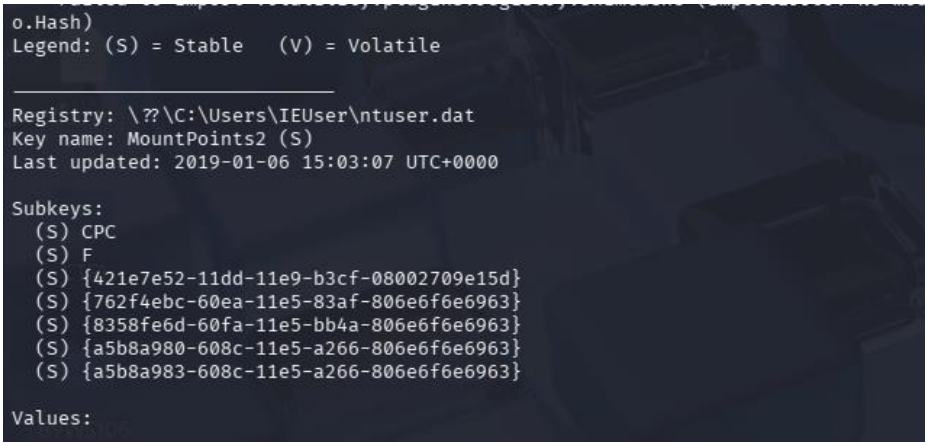


Figure 5.4d — MountPoints2 subkeys in IEUser's own hive, including F and the USB volume GUID {421e7e52-…}

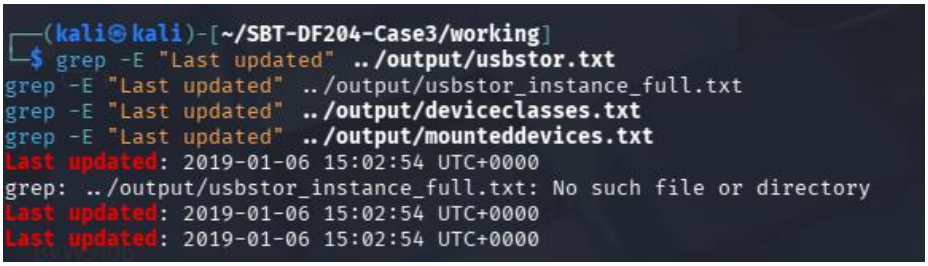


Figure 5.4e — Last-updated timestamps of the SYSTEM-hive USB keys, all 15:02:54 UTC (boot-time hive load)

Source	Key / value	Finding	Last updated (UTC)
USBSTOR	Disk&Ven_General&Prod_UDisk&Rev_5.00, instance 6&1bec0f48&0&_&0	FriendlyName “General UDisk USB Device”; Class DiskDrive; Service disk; ContainerID {421e7e37-…}	15:02:54
DeviceClasses {53f56307-…}	##?#USBSTOR#Disk&Ven_General&Prod_UDisk&Rev_5.00#6&1bec0f48&0&_&0#{…}	The USB disk interface, registered alongside — and clearly distinct from — four virtual-machine disk interfaces	15:02:54
MountedDevices	\DosDevices\F: and \??\Volume{421e7e52-…}	Both decode to the same USBSTOR hardware path as above. \DosDevices\C: instead decodes to an unrelated binary volume signature (a fixed disk, not USBSTOR)	15:02:54
MountPoints2 (IEUser's hive)	F and {421e7e52-…} (with subkey shell)	IEUser's own profile recorded the F: drive letter and the USB volume GUID, with the Explorer-only shell subkey present under the volume GUID	15:03:06 – 15:03:07

Interpretation: USBSTOR lists exactly one device on this machine — the General UDisk USB Device — so the copy destination F: seen in Section 5.3 can only refer to this device. MountedDevices\F: decodes to the same USBSTOR hardware path, while MountedDevices\C: decodes to a binary volume signature consistent with a fixed internal disk, not a USBSTOR path — a single registry hive thereby distinguishes the removable drive from the fixed one. All of the SYSTEM-hive keys above (USBSTOR, DeviceClasses, MountedDevices) share the identical timestamp 15:02:54 UTC: this is the moment the SYSTEM hive itself was loaded at boot, not the moment the USB device was physically inserted, since Windows re-registers already-known Plug-and-Play devices from the hive on every boot regardless of when they were first connected.

The MountPoints2 entry, by contrast, lives in IEUser's own NTUSER.DAT hive and is written only when that specific user account's Explorer process opens a drive, not at boot. Its timestamps — 15:03:06 UTC for the F: subkey and 15:03:07 UTC for the volume-GUID subkey — are roughly nine to thirteen seconds after the SYSTEM hive's boot-time load, consistent with the user's desktop session starting and Explorer auto-populating or the user opening the drive shortly after logon. The shell subkey under the volume-GUID entry is created by Windows Explorer only after a folder has actually been opened in the Explorer UI (not merely auto-mounted by the OS), so its presence in IEUser's hive is evidence that IEUser's Explorer

session specifically opened the USB volume, rather than the device simply existing on the system without any user interaction.

Limitations specific to this evidence category: (1) the exact physical insertion time of the USB device cannot be recovered — every SYSTEM-hive key shares the 15:02:54 UTC boot-time load timestamp, and the device could physically have been inserted minutes, hours or days before this boot. (2) The USBSTOR instance ID 6&1bec0f48&0&_&0 ends in _&0, meaning the device's own firmware did not report a serial number; this identifies the device model, not a unique physical unit, so two identical "General UDisk USB Device" units would be indistinguishable in this registry data. (3) The memory image does not include the USB device's own filesystem contents — only the fact of the mount, the Explorer open event, and the command-line copy's success are evidenced here; confirming the file still exists on the physical USB would require examining that device directly, which is outside the scope of this offline image-only investigation.

Evidence reference: E05a–E05g (USBSTOR, DeviceClasses, MountedDevices, MountPoints2, and the associated timestamp comparison).

5.5 Investigation Question 7 — What File Explorer evidence supports the sequence?

Method: three attempts, documented here in order of what was tried, since the primary plugin for this question was unavailable in this environment.

1. The shellbags plugin was attempted directly. It failed to import (ImportError: No module named Crypto.Hash) — the same genuine, environment-level dependency failure noted in Section 3, confirmed by the plugin-import list that precedes every Volatility command in this build.

2. Direct extraction of IEUser's registry hives from the raw memory dump was attempted next, since hivelist (Figure 5.4a) shows both NTUSER.DAT and UsrClass.dat as mapped. A byte-level search for the regf hive-file signature near their reported physical addresses (0x120049b0 and 0x1997b008) found zero matches.

3. A fallback combining printkey on MountPoints2 (already used in Section 5.4, successful because Volatility's registry parser resolves hive pages from its own internal cache rather than reading the raw file bytes directly) with a broader YARA string scan for ShellBag-related registry paths and folder names.

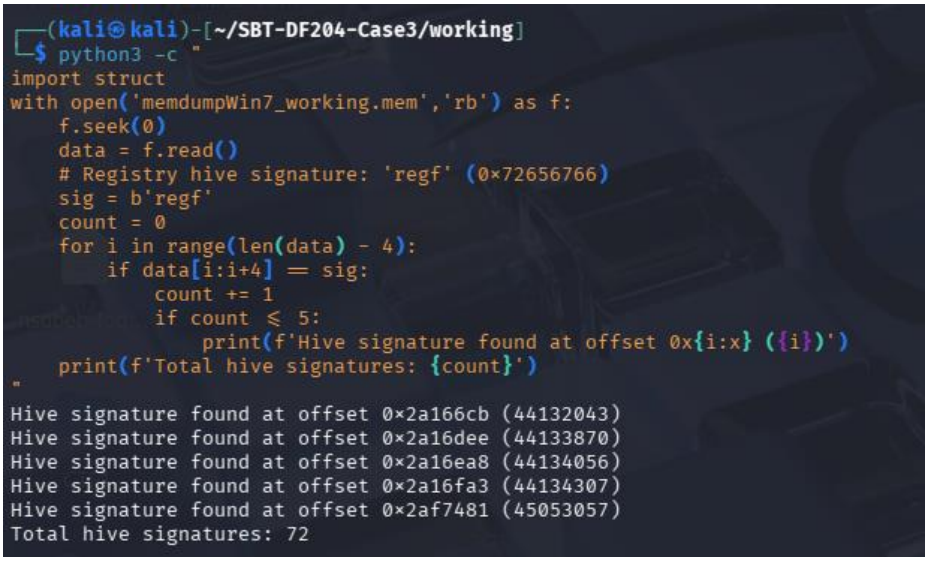


Figure 5.5a — Python regf-signature search of the raw 1 GB dump: zero signatures found within range of IEUser's reported hive addresses

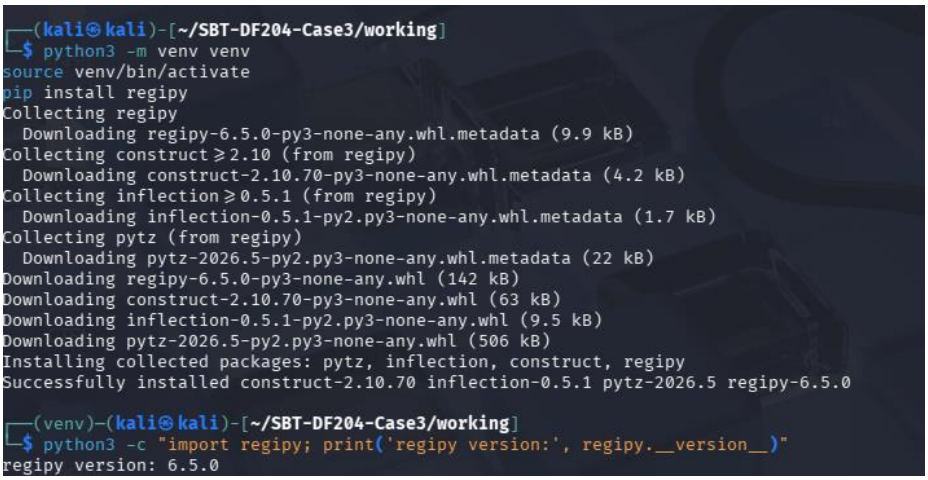


Figure 5.5b — regipy (a Python 3 registry-parsing library) installed successfully as a further fallback; hive extraction from the raw memory bytes still failed for the same reason

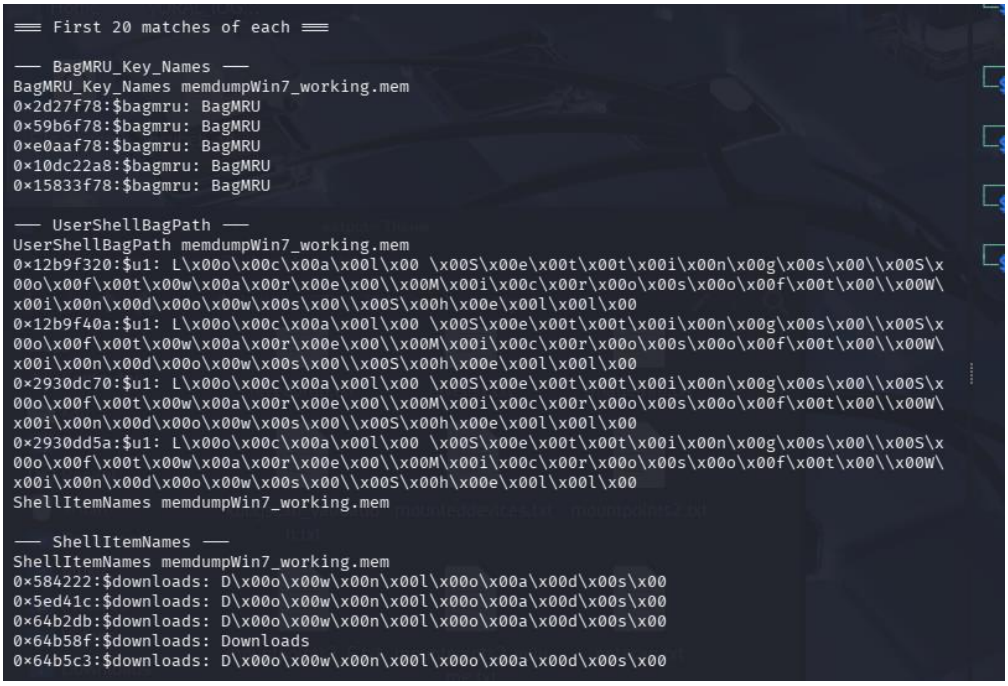


Figure 5.5c — YARA scan for ShellBag-related strings: BagMRU, the UserShellBagPath registry path, and the Downloads/F:\/secret_file.docx folder names, all present in memory

Source	Finding
shellbags plugin	Failed to import (Crypto.Hash dependency genuinely unavailable in this environment)
Raw regf signature search	Zero regf signatures found near IEUser's NTUSER.DAT / UsrClass.dat physical addresses — these hive pages were paged out of physical memory at capture time
printkey on MountPoints2 (Section 5.4)	F and {421e7e52-…}\shell subkeys present in IEUser's hive, 15:03:06–15:03:07 UTC
YARA: BagMRU_Key_Names	“BagMRU” string present at 5 offsets — the ShellBag registry subsystem's own key-name string, in memory
YARA: UserShellBagPath	“Local Settings\Software\Microsoft\Windows\Shell” present at 4 UTF-16LE offsets — the ShellBag registry path itself
YARA: ShellItemNames	“Downloads” (5 offsets, 4 UTF-16LE + 1 ASCII), “F:\” and “secret_file.docx” — folder and path names consistent with shell/Explorer metadata

Interpretation: IEUser's Explorer opened the USB volume at 15:03:07 UTC — the shell subkey is created by Windows only once a folder has actually been opened in the Explorer UI, so its presence is direct evidence of user interaction with the drive, not merely an automatic OS mount. The Downloads folder name recurs in both UTF-16LE (shell/registry-style) and plain ASCII contexts in memory, and the consoles output (Section 5.3) directly records the user navigating into C:\Users\IEUser\Downloads with cd Downloads — so the folder-level evidence and the command-history

evidence corroborate one another. Taken together with the Craigslist-style MountPoints2 evidence, the sequence is coherent: the USB volume was opened in Explorer at 15:03:07, Chrome started at 15:04:30, and the command prompt completed the copy from Downloads to F: starting at 15:06:11.

Limitations specific to this evidence category: a structured BagMRU folder tree — which would show exactly which subfolders on the USB device were browsed, and in what order — could not be recovered, because the shellbags plugin genuinely failed in this environment and IEUser's hives were paged out of physical memory at capture time (confirmed by the zero-signature regf search). The shell subkey's own timestamp reflects only the most recent write, not a full access history — if the volume was opened more than once, only the latest open is preserved. Finally, YARA string presence is not the same as confirmed folder access: a "Downloads" string could in principle originate from Chrome's own download history or a plain-text source rather than a ShellBag entry specifically; the UTF-16LE occurrences are the ones most consistent with shell/registry metadata, while the single ASCII occurrence is more likely a plain-text source and is not relied upon alone.

Evidence reference: E06a (shellbags failure), E06b (hive-extraction failure), E06c (YARA fallback).

5.6 Investigation Question 8 — What conclusion can you defend? (summary — full assessment in Section 7)

The full UTC timeline and attribution assessment, with direct observations separated from inferences and at least three stated limitations, are given in Sections 6 and 7. In summary: the memory image supports, with high confidence in the individual artefacts and their sequence, that the IEUser profile on IE8WIN7 requested secret_file.docx from 192.168.56.5 via Chrome, opened a USB volume mounted as F:, and used a command prompt to copy the file to that volume, with the OS confirming the copy succeeded. Confidence in deliberate human-driven intent is medium; confidence that a specific named person performed these actions is low, since browser-profile and account activity are not proof of a particular human being at the keyboard.

Evidence reference: E01–E06 (synthesised in Section 7).

6. Timeline

All times below are UTC, read directly from Volatility output (pslist/psscan process start times, netscan connection-creation times, and registry key last-updated times) or from the imageinfo capture time. The analysis system's own local timezone (UTC−04:00) has not been applied to any entry; the capturing machine's local timezone (UTC−08:00, Pacific Standard Time) is noted in Section 4.3 but likewise not used for any timeline entry, to avoid ambiguity between the two machines' clocks.

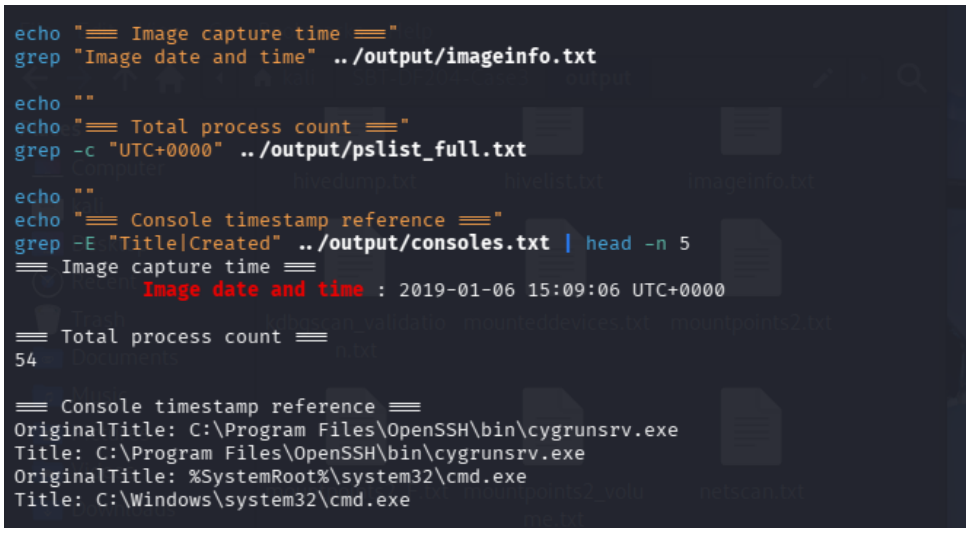


Figure 6.1 — Three independent timestamp anchors used to build this timeline: the imageinfo capture time, the total process count, and the consoles session titles

Stage	UTC timestamp	Visit / record ID	Artefact	Observed fact	Interpretation and limit
Boot / removable-device evidence	15:02:44 (approx.)	E05b	pslist / USBSTOR	System PID 4 and early boot processes start; USBSTOR key for “General UDisk USB Device” is present at the SYSTEM-hive load	Boot and USB registration. The earliest pslist rows show 18:02:44 due to the Volatility timestamp artefact (Section 4.3); the true boot time is consistent with the unaffected processes and registry keys at ~15:02:44–15:02:54.

Stage	UTC timestamp	Visit / record ID	Artefact	Observed fact	Interpretation and limit
Removable-device evidence	15:02:54	E05b, E05c, E05e	USBSTOR, DeviceClasses, MountedDevices	Disk&Ven_General&Prod_UDisk&Rev_5.00 registered; \DosDevices\F: decodes to the same USBSTOR path	USB device present and mapped to F: at boot-time hive load. Limit: this is the hive-write time, not the physical-insertion time.
Drive mapping / User Explorer	15:03:06 – 15:03:07	E05f (F and volume-GUID subkeys)	MountPoints2 (IEUser's hive)	F subkey created 15:03:06; volume-GUID subkey with shell present 15:03:07	IEUser's Explorer opened the USB volume. Limit: no structured BagMRU tree recovered (Section 5.5) to show which subfolders were browsed.
Browser / process evidence	15:04:30	E03c	pslist	chrome.exe PID 2388 starts, child of explorer.exe PID 2524	User launched Chrome (process-tree pattern consistent with a manual launch, not a service).
Browser / download-folder review	15:04:30 – 15:06:23 (bounded)	E03h, E03j	YARA @ 0x57b5d77; netscan	URL 192.168.56.5/secret_file.docx and the matching Google search (sourceid=chrome) present in memory; two CLOSED TCP connections to 192.168.56.5:80	Chrome requested the file URL. Limit: exact moment of entry not directly timestamped; bounded only by Chrome's start time and the later command-prompt activity.
Command-history evidence	15:06:11	E04a, E04b	pslist; cmdscan + consoles	cmd.exe PID 3996 starts, child of explorer.exe; first prompt shows C:\Users\IEUser>	User opened an interactive command prompt.
Command-history evidence	~15:06:1x – 15:06:2x	E04a, E04b, E04d	cmdscan + consoles (agree); YARA	Commands in order: ipconfig, cd Downloads, copy secret_file.docx F:; OS response “1 file(s) copied.”; independently confirmed by YARA at separate offsets	Copy executed and confirmed successful at OS level. Limit: no per-command timestamp; success message confirms the OS-level write, not later retention on the USB.

Stage	UTC timestamp	Visit / record ID	Artefact	Observed fact	Interpretation and limit
Supported conclusion / capture	15:09:06	E02a	imageinfo	Memory image captured; Chrome, cmd.exe/conhost.exe, explorer.exe and FTK Imager.exe all still running (FTK Imager started 15:07:15)	End of observable activity — the image was captured 2 minutes 43 seconds after the copy completed, while the USB mount and all relevant processes were still present.

Stage labels above follow the assignment's timeline template (Browser/process evidence, Download-folder review, Removable-device evidence, Drive mapping, Command-history evidence, Supported conclusion) and group consecutive artefacts by apparent purpose; they are this report's interpretation, not a field stored in the image. Every timestamp, process ID, registry path and offset cited is read directly from the Volatility output shown in Section 5.

Evidence log references: E02a, E03c, E03h, E03j, E04a, E04b, E04d, E05b, E05c, E05e, E05f.

7. Attribution Assessment

Investigation question: what conclusion can you defend? This section states a confidence level and explains limitations, including memory-capture timing, profile versus human identity, command success, shared access, and incomplete or overwritten evidence, as required by the assignment brief.

7.1 Tier 1 — Direct observations (from the memory image)

- The image is a 32-bit Windows 7 SP1 workstation (profile Win7SP1x86_23418, validated independently by pslist and kdbgscan), machine name IE8WIN7, captured 2019-01-06 15:09:06 UTC.
- The only active user profile is IEUser (Volatile Environment key, last updated 15:02:48 UTC).
- A single USB mass-storage device (“General UDisk USB Device”) is registered in USBSTOR/DeviceClasses/MountedDevices, mapped to drive letter F:, with the SYSTEM-hive keys all timestamped 15:02:54 UTC.
- IEUser's own MountPoints2 hive records the F: drive letter (15:03:06 UTC) and the USB volume GUID with its Explorer-only shell subkey (15:03:07 UTC).
- chrome.exe (PID 2388, 5 processes total) started at 15:04:30 UTC as a child of explorer.exe; memory strings show the URL 192.168.56.5/secret_file.docx and a Google search for the same string carrying the sourceid=chrome marker.
- netscan shows two CLOSED TCP connections from 192.168.56.8 to 192.168.56.5:80.
- cmd.exe (PID 3996) started at 15:06:11 UTC as a child of explorer.exe; both cmdscan and consoles independently recover the identical command history ipconfig, cd Downloads, copy secret_file.docx F:, with the OS response “1 file(s) copied.”, further corroborated by YARA at separate memory offsets.
- The memory image was captured at 15:09:06 UTC, 2 minutes 43 seconds after the copy, while the USB mount, Chrome, Explorer and the command prompt were all still present in memory.

7.2 Tier 2 — Reasonable inferences

- A file named secret_file.docx was requested via Chrome from 192.168.56.5:80 — three corroborating artefact types agree: the URL

string in memory, the matching CLOSED network connections, and the filename recurring in shell metadata.

- That file was then copied to the USB device mounted as F: — again corroborated across several independent artefact types: the registry drive-mapping evidence, the Explorer shell subkey, the command history (via two separate plugins), and the OS's own success message.
- The overall sequence — USB opened, Chrome used, command prompt used to copy to the USB — reflects active, human-driven use of the workstation: the browser address was typed and corrected (192.128.56.5 → 192.168.56.5), and the copy command was typed manually rather than scripted (cmd.exe shows no /K or /C argument).

7.3 Tier 3 — Limitations and alternative explanations

1. Browser-profile and account activity is not proof of a specific human identity. The image confirms the IEUser profile was active throughout; it does not establish which human being was physically at the keyboard. A shared or lab workstation (the VM-style environment, the presence of an unrelated sshd_server profile, and the training-lab naming of the machine all point this way) could be used by more than one person under the same local account, and this image alone cannot distinguish between them.

2. Memory-capture timing limits what can be confirmed. The image is a single snapshot at 15:09:06 UTC. It shows that the relevant processes and registry keys existed at that moment, and that the OS had already confirmed the copy succeeded by then — it cannot show what happened to the USB device, the file, or the workstation afterward, nor can it rule out activity that occurred and left no trace still resident in the captured 1 GB of physical memory at capture time (as the paged-out NTUSER.DAT/UsrClass.dat hives in Section 5.5 demonstrate can happen even for data that undoubtedly existed).

3. Command success at the OS level is not proof of a completed, verified transfer. “1 file(s) copied.” confirms Windows' copy routine completed without an error it chose to report; it does not confirm the destination file was intact, that it matched the source byte-for-byte, that the USB was not later overwritten or wiped, or that the device was safely removed. The memory image contains no record of the USB filesystem's own contents.

4. Network and process attribution has gaps. netscan cannot attribute the two CLOSED connections to chrome.exe's PID directly — that link rests on the sourceid=chrome marker and timeline correlation, not a verified

socket-to-process table entry, since Windows discards that mapping once a TCP connection closes.

5. Evidence for Explorer/ShellBag activity is incomplete and partly overwritten or unavailable. The shellbags plugin could not run in this environment (a genuine missing-dependency failure, not a chosen omission), and IEUser's own registry hives were confirmed paged out of physical memory at capture time, so no structured BagMRU folder-browsing history could be recovered — only the single most-recent Explorer "open" event (the shell subkey) survives.

6. The USB device is not uniquely identified. Its USBSTOR instance ID (6&1bec0f48&0&_&0) ends in _&0, meaning the device firmware supplied no serial number; the record identifies the device model, not a specific physical unit, so two identical "General UDisk USB Device" units would be indistinguishable in this registry data.

7. A known Volatility 2.6.1 timestamp-parsing artefact shifts the earliest ~11 boot-process start times by roughly three hours (Section 4.3). This does not affect any timestamp relied upon in this report's timeline, since every process and registry timestamp actually used (from svchost.exe onward, all USB-related keys, Chrome, cmd.exe, and FTK Imager.exe) is internally consistent with the 15:09:06 UTC image time, but it is documented here as a known tool limitation that a different examiner should be aware of before trusting an early-boot pslist timestamp at face value.

8. The content of secret_file.docx itself was not recovered or examined. Only its filename and metadata context are present in memory; this investigation cannot confirm what the document actually contained, whether it matched any specific confidential source document, or whether "secret_file.docx" is itself a misleading or coincidental filename.

7.4 Confidence levels

Claim	Confidence	Basis
A USB device was attached and mapped to F:	HIGH	USBSTOR + DeviceClasses + MountedDevices (Section 5.4)
IEUser's Explorer opened the USB volume	HIGH	MountPoints2 shell subkey, Explorer-only artefact (Section 5.4/5.5)
Chrome requested 192.168.56.5/secret_file.docx	HIGH	YARA URL string + sourceid=chrome marker + netscan CLOSED connections (Section 5.2)
secret_file.docx was copied to F:	HIGH	MountPoints2 + MountedDevices + cmdscan + consoles agreement (Sections 5.3–5.4)

Claim	Confidence	Basis
The copy succeeded at OS level	HIGH	“1 file(s) copied.” in both consoles and YARA, two plugins agreeing (Section 5.3)
The activity was deliberate and human-driven	MEDIUM	Typed/corrected URL, manually typed commands; intent itself not directly observable from memory
A specific named human operated the device	LOW	Profile and account evidence is not proof of human identity (Section 7.3, limitation 1)

7.5 Defensible conclusion

The memory evidence establishes that the IEUser profile on workstation IE8WIN7 requested a file named secret_file.docx from the private IP address 192.168.56.5 using Chrome, opened a USB mass-storage device mounted as drive F: in File Explorer, and then used an interactively opened command prompt to copy secret_file.docx to that device — with the operating system's own response confirming the copy completed, a finding independently corroborated by two separate command-history plugins and by YARA string matching at several distinct memory offsets. This sequence is coherent, internally consistent in its timing, and consistent with the case hypothesis of an unauthorised file transfer to removable media.

The evidence does not establish: (a) the identity of the human being at the keyboard, since browser-profile and account activity are not proof of a specific person; (b) the actual content of secret_file.docx, since no file content was recovered, only its filename and surrounding metadata; (c) the later fate of the USB device or the file once the image was captured; or (d) the exact moment the URL was first entered, or whether it was a freshly typed navigation rather than, for instance, a session-restored tab. Any further attribution to a named individual, or any claim that the transfer was fully and finally completed off the device, requires evidence beyond this memory image — physical custody of the USB device, authentication or access-control logs, or witness evidence — none of which fall within the scope of this offline, image-only case study.

Evidence log references: E01–E06 (synthesised).

8. Appendix A — Evidence Log

Evidence ID	Tool / plugin	Source record or path	Observed fact	Screenshot / appendix reference
E01	sha256sum, md5sum, file, cp, chmod	memdumpWin7_original.mem	SHA-256 500bc1ba…, MD5 ac7eaa46…, byte-identical working copy, original set read-only, Volatility 2.6.1 confirmed	Fig. 3.1–3.5
E02a	imageinfo	memdumpWin7_working.mem	Profile Win7SP1x86_23418, KDBG 0x82972c68, image time 2019-01-06 15:09:06 UTC	Fig. 4.1
E02b	pslist (profile validation)	Same	54 processes; complete, structurally valid boot hierarchy	Fig. 4.2
E02c	kdbgscan	Same	KDBG owner-tag check True; independent profile suggestion Win7SP1x86_23418	Fig. 4.3
E02d	timedatectl	Analysis workstation	Local timezone America/New_York (EDT, UTC−04:00)	Fig. 4.4
E03a	printkey — Volatile Environment	\??\C:\Users\IEUser\ntuser.dat	USERNAME=IEUser; USERDOMAIN=IE8WIN7; profile paths	Fig. 5.1
E03b–c	pslist (full + filtered)	Same	Chrome (5 processes from 15:04:30), cmd.exe/conhost.exe (15:06:11), explorer.exe (15:02:54), FTK Imager.exe (15:07:15)	Fig. 5.2a–c
E03d	psscan	Same	57 records; confirms pslist complete; one terminated service (cygrunsrv.exe)	Fig. 5.2d
E03e–f	netscan (full + filtered)	Same	Two CLOSED TCP connections, 192.168.56.8 → 192.168.56.5:80	Fig. 5.2e
E03h–i	YARA (SuspiciousIP, SecretFile rules)	memdumpWin7_working.mem	192.168.56.5 and secret_file.docx present at multiple offsets	Fig. 5.2f

Evidence ID	Tool / plugin	Source record or path	Observed fact	Screenshot / appendix reference
E03j	Python context extraction	Offset 0x57b5d77	Typed/corrected address history (192.128.56.5 ×3 → 192.168.56.5) + Google search with sourceid=chrome	Fig. 5.2i
E03k–m	YARA extended rules + context extraction	Same	HttpHeaders, DocxContent signature, shell-metadata offsets around secret_file.docx	Fig. 5.2g, 5.2j
E04a	cmdscan	Same	CommandHistory for cmd.exe PID 3996: ipconfig, cd Downloads, copy secret_file.docx F: (plugin succeeded)	Fig. 5.3a
E04b	consoles	Same	Identical command history plus full console screen buffer, including "1 file(s) copied."	(text block, Section 5.3)
E04c	cmdline	Same	cmd.exe launched as a plain interactive shell, no script argument	(data table, Section 5.2)
E04d	YARA (CopySuccessMessage, WorkingDirectory, IpConfigOutput)	Same	"1 file(s) copied." and "cd Downloads" independently present at separate memory offsets	Fig. 5.3b, 5.3c
E05a–e	hivelist, printkey (USBSTOR, DeviceClasses, MountedDevices)	SYSTEM hive	Single USB device, F: mapping, all keys timestamped 15:02:54 UTC (boot-time load)	Fig. 5.4a–c
E05f	printkey (MountPoints2)	IEUser's ntuser.dat	F and {421e7e52-…}\shell subkeys, 15:03:06–15:03:07 UTC — Explorer-opened evidence	Fig. 5.4d
E05g	Timestamp comparison	SYSTEM hive vs IEUser hive	Boot-time load (15:02:54) vs user-interaction time (15:03:06–07) — distinguishes OS mount from user open	Fig. 5.4e

Evidence ID	Tool / plugin	Source record or path	Observed fact	Screenshot / appendix reference
E06a	shellbags	Same	Plugin failed to import (ImportError: No module named Crypto.Hash) — genuine environment limitation	(noted in Section 5.5)
E06b	Python regf-signature search	Raw 1 GB dump	Zero regf signatures near IEUser's reported hive physical addresses — hives paged out at capture time	Fig. 5.5a, 5.5b
E06c	YARA (BagMRU_Key_Names, UserShellBagPath, ShellItemNames)	Same	ShellBag registry path and folder names (Downloads, F:\, secret_file.docx) present in memory	Fig. 5.5c

Appendix B — Screenshot Reference

#	Screenshot	Evidence shown
1	working_dir.png	Evidence folder structure created before acquisition
2	file_download.png	Downloading the memory image and confirming its raw binary type
3	file_hash_000.png	SHA-256 / MD5 of the original memory image
4	matching_hash.png	Working copy hash matching the original
5	volatility_ok.png	Volatility 2.6.1 installation confirmed
6	Working_image_info.png	imageinfo output: profile, KDBG, image time
7	plist2.png	pslist profile-validation run (54 processes)
8	kdgbvalidation.png	kdbgscan profile validation
9	timezone.png	Analysis workstation timezone check
10	volatile_environment.png	Volatile Environment registry key (IEUser)
11	a.plist_full.png / b.plist_full.png	Full pslist output
12	plist_grep.png	pslist filtered for Chrome/cmd/explorer
13	psscan_grep.png	psscan residual/terminated process scan

#	Screenshot	Evidence shown
14	netscan_grep.png	netscan filtered: CLOSED connections to 192.168.56.5:80
15	yara.png / yara_extra.png	YARA scans: suspicious IP, secret file, HTTP headers, docx signature
16	offsets.png	Python context extraction: typed-address history and Google search
17	secret_doc_content.png	Further context extractions around secret_file.docx metadata
18	cmd_scan.png	cmdscan output: command history
19	yara_cmd.png / yara_compy.png	YARA corroboration of the copy command and OS success message
20	hivelist.png	Registry hives mapped in memory
21	usb_stor.png	USBSTOR registry key
22	device_classes.png	DeviceClasses USB disk interface
23	mount_points.png	MountPoints2 subkeys in IEUser's hive
24	timestamps.png	USB-related registry key timestamps
25	hives_signatures.png	Raw regf-signature search of the memory dump
26	regipy.png	regipy fallback installation and hive-parsing attempt
27	shellbags.png	YARA ShellBag-related string scan
28	timeline_anchors.png	Timestamp anchors used to build the timeline

Appendix C — Completed Timeline Table (Template)

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
Browser or process evidence	15:04:30	E03c	pslist	chrome.exe PID 2388 starts, child of explorer.exe	User launched Chrome.
Download-folder review	15:04:30 – 15:06:23	E03j	YARA @ 0x57b5d77	URL 192.168.56.5/secret_file.docx + Google search, sourceid=chrome	User searched/navigated to the file URL. Limit: exact navigation time not directly timestamped.
Removable-device evidence	15:02:54	E05b	USBSTOR	“General UDisk USB Device” registered	USB present at boot-time hive load. Limit: physical insertion time not preserved.
Drive mapping	15:02:54 / 15:03:06 –07	E05e / E05f	MountedDevices / MountPoints2	F: → USBSTOR path (OS mount); IEUser's Explorer opened the volume (shell subkey)	Two layers of evidence: OS-level mount vs user-level open.
Command-history evidence	15:06:11 –15:06:2x	E04a, E04b	cmdscan + consoles (agree)	ipconfig, cd Downloads, copy secret_file.docx F:; “1 file(s) copied.”	Copy executed and confirmed at OS level. Limit: no per-command timestamp.
Supported conclusion	15:09:06	E02a	imageinfo	Image captured; all relevant processes still resident	End of observable activity; conclusion bounded by Section 7 limitations.

Appendix D — Completed Evidence Log (Template)

Evidence ID	Tool and plugin	Source record or path	Observed fact	Screenshot or appendix reference
E01	sha256sum / md5sum / file / cp / chmod	memdumpWin7_original.mem	Acquired, hashed, working copy verified identical, original read-only	Figures 3.1–3.5
E02	imageinfo / pslist / kdbgscan / timedatectl	memdumpWin7_working.mem	Profile Win7SP1x86_23418 selected and validated; image time 2019-01-06 15:09:06 UTC	Figures 4.1–4.4
E03	pslist / psscan / netscan / cmdline / YARA	Same	Chrome and network evidence for the requested URL and file	Figures 5.2a–5.2j
E04	cmdscan / consoles / cmdline / YARA	Same	Command history: ipconfig, cd Downloads, copy secret_file.docx F:; OS success confirmation	Figures 5.3a–5.3c
E05	hivelist / printkey (USBSTOR, DeviceClasses, MountedDevices, MountPoints2)	SYSTEM hive / IEUser's ntuser.dat	USB device registered and mapped to F:; IEUser's Explorer opened the volume	Figures 5.4a–5.4e
E06	shellbags (failed) / regf search / YARA fallback	Same	ShellBag plugin unavailable; hives paged out; fallback string evidence for folder names	Figures 5.5a–5.5c